T3000 API & Protocol

Status Bar Component · built Nov 13, 2026

Status Bar Component

Overview

The Status Bar sits at the bottom of the T3000 application window and shows real-time feedback about the system state. It reports RX/TX packet counts and displays color-coded status messages to keep you informed about what the system is doing.

What You See

The Status Bar has two visible sections:

RX/TX Statistics

Shows how many data packets have been received (**RX**) and transmitted (**TX**) between your browser and the backend server. These counters increment as the system communicates.

Status Message

Displays a text message describing the current operation or result. The message bar changes color depending on the type of information:

Color	Meaning	Example
Gray	Informational	`Refreshing variables from device...`
Green	Success	`✓ Synced 42 variables from Panel-1`
Yellow	Warning	`No devices in database`
Red	Error	`Failed to sync: Network error`

If a message is longer than 100 characters, an **info icon** () appears next to the message. Hover over it to see the full text in a tooltip.

Where Messages Come From

Messages appear automatically when the system performs actions. Here are common scenarios:

- **Loading a device** → `Loading variables from Panel-1 (Action 17)...`

- **Refresh complete** → `✓ Synced 50 inputs from Panel-1`
- **Database operations** → `Loaded 15 devices`
- **Errors** → `Failed to sync: Connection refused`

You do not need to interact with the Status Bar — it updates automatically as you use the application.

Component Architecture

File Locations

File	Purpose
`src/t3-react/layout/StatusBar.tsx`	Presentational component (Fluent UI styled)
`src/t3-react/store/statusBarStore.ts`	Zustand state management
`src/t3-react/layout/MainLayout.tsx`	Wires store → component via props

Props Interface

TYPESCRIPT

```typescript
export interface StatusBarProps {
  rxCount?: number;        // Received packet count (default: 0)
  txCount?: number;        // Transmitted packet count (default: 0)
  buildingName?: string;   // Building/panel name (default: '')
  deviceName?: string;     // Device name (default: '')
  protocol?: string;       // Protocol name (default: '')
  connectionType?: string; // Connection type (default: '')
  message?: string;        // Status message text (default: 'Ready')
  messageType?: MessageType; // 'info' | 'success' | 'error' | 'warning' (default: 'info
}
```

> **Note:** Panes for Building/Device and Protocol info are currently **commented out** in the JSX. Only RX/TX stats and the message pane are active.

Zustand Store API

TYPESCRIPT

```typescript
type MessageType = 'info' | 'success' | 'error' | 'warning';

interface StatusBarState {
  // State fields
  rxCount: number;
  txCount: number;
  buildingName: string;
  deviceName: string;
  protocol: string;
  connectionType: string;
  message: string;
  messageType: MessageType;

  // Actions
  incrementRx: () => void;
  incrementTx: () => void;
  setRxTx: (rx: number, tx: number) => void;
  setConnection: (building: string, device: string) => void;
  setProtocol: (protocol: string, connectionType: string) => void;
  setMessage: (message: string, type?: MessageType) => void;
  reset: () => void;
}
```

How to Dispatch Messages

From inside a React component (reactive hook):

TYPESCRIPT

```typescript
import { useStatusBarStore } from '@t3-react/store/statusBarStore';

const setMessage = useStatusBarStore((state) => state.setMessage);

// Informational
setMessage('Refreshing variables from device...', 'info');

// Success
setMessage('✓ Synced 42 variables from Panel-1', 'success');

// Warning
setMessage('No devices found in T3000', 'warning');

// Error
setMessage('Failed to sync: Network error', 'error');

// type defaults to 'info' if omitted
setMessage('Loading data...');
```

From outside React (e.g., stores, services, callbacks):

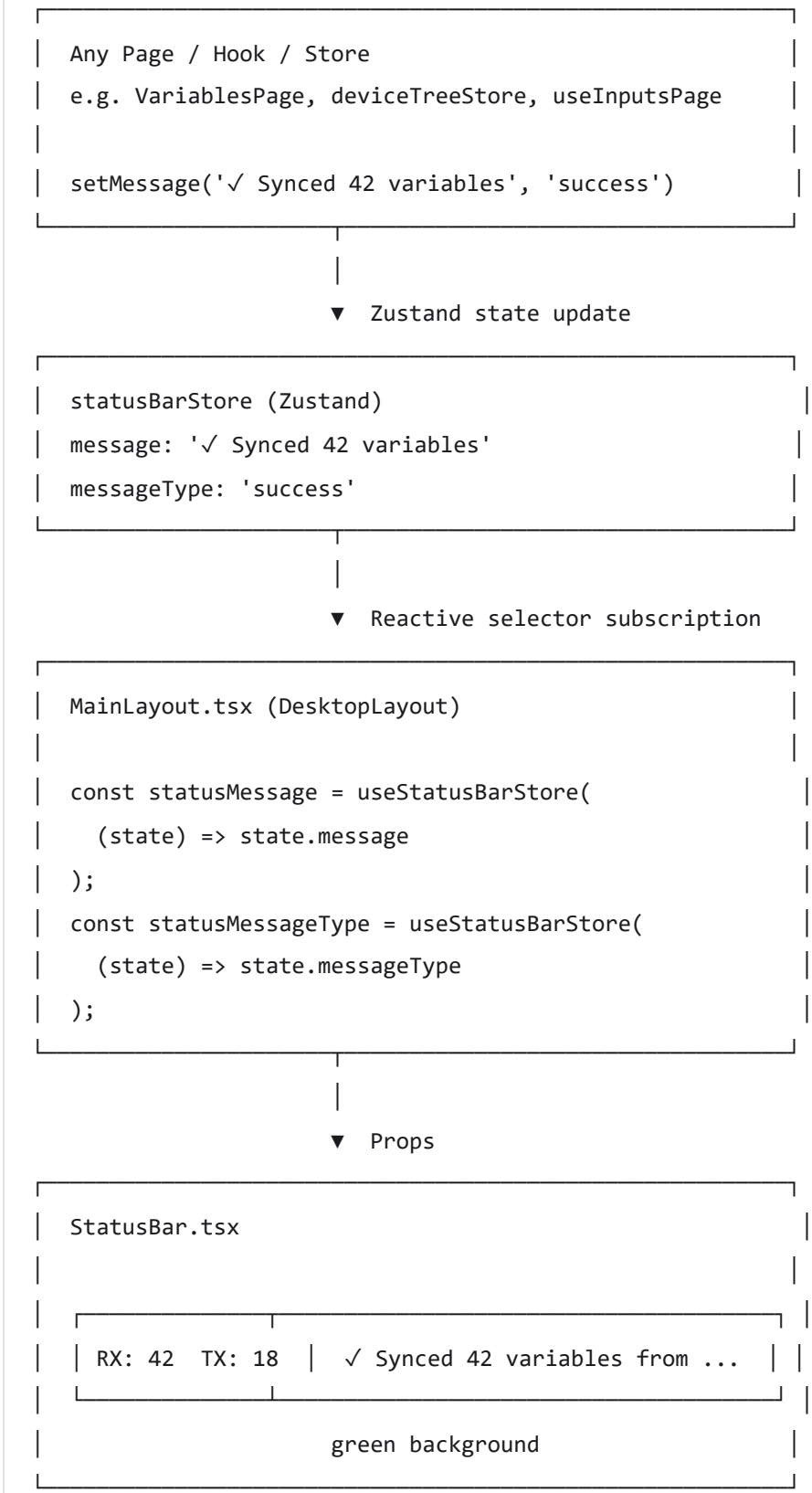
TYPESCRIPT

```typescript
import { useStatusBarStore } from '@t3-react/store/statusBarStore';

useStatusBarStore.getState().setMessage('Loaded 15 devices', 'success');
useStatusBarStore.getState().setMessage('Error: Connection refused', 'error');
```

Data Flow Diagram

TEXT



Message Type Styling

The message type controls both the background color of the entire status bar and the text color of the message:

`messageType`	Bar Background	Text Color	CSS Class
`'info'`	`#f0f0f0` (gray)	`#323130` (default)	—
`'success'`	`#dff6dd` (green)	`#0e700e`	`statusBarSuccess` / `messagePaneSuccess`
`'error'`	`#fde7e9` (red)	`#a4262c`	`statusBarError` / `messagePaneError`
`'warning'`	`#fff4ce` (yellow)	`#8a5d00`	`statusBarWarning` / `messagePaneWarning`

Long Message Tooltip

When `message.length > 100`, a Fluent UI `Tooltip` wraps an `Info16Regular` icon next to the message text. Hovering shows the full message positioned `above-start`.

Key Consumers

The status bar receives messages from across the application:

- **Device tree** (`deviceTreeStore.ts`) — device load, sync, delete operations
- **Variables page** (`useVariablesPage.ts` , `VariablesPage.tsx`) — fetch, refresh, sync
- **Inputs page** (`useInputsPage.ts` , `InputsPage.tsx`) — fetch, refresh, sync
- **Outputs page** (`useOutputsPage.ts` , `OutputsPage.tsx`) — fetch, refresh, sync
- **Graphics page** (`GraphicsPage.tsx`) — graphic load, sync
- **Tree toolbar** (`TreeToolbar.tsx`) — device refresh errors

Design Notes

- **Global singleton** — One Zustand store; any code path can write messages without prop drilling.
- **No auto-dismiss** — Messages persist until overwritten by another call or `reset()` is invoked.
- **Reactive rendering** — Zustand selectors ensure StatusBar re-renders only when the subscribed fields change.
- **Thread-safe** — `useStatusBarStore.getState()` allows non-React code (stores, callbacks, services) to dispatch messages safely.

Document generated for T3000 Webview v9.0 — Status Bar Component